

PHOENICIAN CAPITAL · LOCAL-ENGINE-INTERNET

Live demo – Search + DeepSeek Flash

A short paper you can send. How the service works, what we tested on 9 September 2026, and the live numbers DeepSeek Flash produced after searching and reading Apple’s own filings. How to run it is at the end. No cloud writing models. Same Flash you will run in the datacenter.

Engine: `http://127.0.0.1:8090`

Model: `deepseek-v4-flash` (DeepSeek API today · local Brain tomorrow)

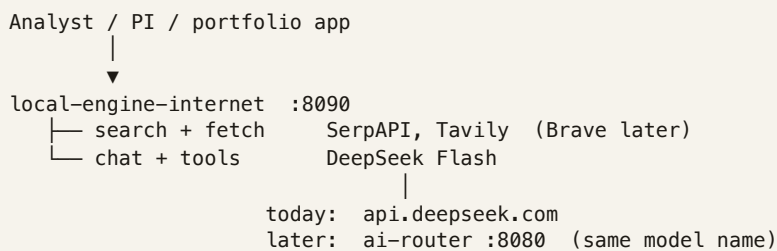
Search: SerpAPI + Tavily

Date of run: 9 September 2026, 10:50-10:55 UTC

Internals (full code map): `Local-LLM-Internet-Engine-Method.pdf`

1. How it works

Company apps already talk to DeepSeek with `POST /v1/chat/completions` (messages, tools, `tool_calls`). This service sits on port `8090` and speaks that same DeepSeek / vLLM wire format. It does not load model weights. It searches the web, downloads pages, and — in chat — sits in front of DeepSeek Flash.



Mode 1 — the pipeline already decided to search. `POST /v1/search`, `/v1/fetch`, `/v1/research`. No model in the hop.

Mode 2 — the user sends a question. Flash may call `web_search` and `fetch_url`. The engine runs those tools and asks Flash again until it answers in prose. Sources stay labeled. If search is required and fails, the HTTP status is 424 — it does not invent numbers from training data.

When the datacenter Brain is up, change one setting: `UPSTREAM_LLM_BASE_URL=http://127.0.0.1:8080`. Apps keep pointing at `http://127.0.0.1:8090/v1`. You stop using Claude / GPT / Gemini / Perplexity as the writer. You still use search APIs (Google via SerpAPI, Tavily). Those are indexes, not models.

2. What we tested

Two batteries on the same machine, same engine process (restarted only to point chat at DeepSeek’s API).

Check	What it proves	Result
Health / plugin	Service up, internet plugin on, model may choose when to search	Pass
SerpAPI + Tavily mix	Both indexes run in parallel; one failure does not kill the request	Pass – 0 errors
GET /search	SerpAPI-shaped JSON for existing scrapers	Pass
Fetch HTML	example.com and investor.apple.com	Pass
Fetch official 10-Q PDF	Parser reads Apple’s IR PDF (income statement)	Pass
Fail-closed (no Brain)	Chat while :8080 is down	502 – search still works; no invented answer

Check	What it proves	Result
Tool-call canary (DeepSeek API)	Flash emits structured tool_calls	Pass
Mode 2, policy required	Flash must search, then answer with URLs	200 in 77.6s
Mode 2, policy off	"What is 2+2?" stays offline	"4" – no tools, 1.1s
pytest (71 tests)	No live keys; locks mix, plugin, fail-closed, fetch ladder	Pass (run separately)

Brave was not configured. It is optional — the mix runs without it. Tavily returned 403 earlier in the day and 200 on retry; the mix skipped the failure and kept SerpAPI.

3. Live Mode 1 – search and fetch

Ran 10:50–10:51 UTC. No model. Query used for the clean hit list:

AAPL 10-Q "June 27, 2026" OR aapl-20260627 site:sec.gov OR site:q4cdn.com

	Value
HTTP	200
Time	3.7 seconds
Providers used	serpapi, tavily
Errors	none
Hits	8
Cross-confirmed URL (both indexes)	https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/aapl-20260627.htm

Tavily also returned Apple's 30 July 2026 newsroom / 8-K and the FY26 Q3 statements PDF. A looser query ("Apple Inc latest 10-Q...") still returned both providers but mixed in junk (wrong CIK, unrelated video). The engine does not hide bad hits — Flash (or an analyst) must weigh them. That is intentional.

Official PDF we fetched ourselves

POST /v1/fetch of Apple's IR copy of the Q2 FY2026 10-Q (quarter ended 28 March 2026), 3.8s, httpx, no browser:

https://s2.q4cdn.com/470004039/files/doc_earnings/2026/q2/filing/10Q-Q2-2026-as-filed.pdf

Line extracted from the PDF	Q2 FY26	Prior-year quarter
Total net sales	111,184	95,359
Operating income	35,885	29,589
Net income	29,578	24,780
Diluted EPS	2.01	1.65

These figures came out of the PDF parser, not a model. Q2 ≠ Q3. The later chat answer is Q3 (June quarter). Keeping them separate is the point of labeled sources.

4. Live Mode 2 – DeepSeek Flash with tools

Ran 10:54 UTC. Engine on :8090. Flash at <https://api.deepseek.com>, model deepseek-v4-flash. Same name you will load on vLLM.

Prompt we sent

Apple Inc (AAPL) most recently reported quarterly revenue, operating income, and diluted EPS. Prefer the official 10-Q or earnings release on SEC EDGAR or investor.apple.com. Return the filing URL.

Body extra: phoenician_web.policy: "required" — Flash must complete a successful search or the engine returns 424.

	Value
HTTP	200
Wall time	77.6 seconds
Tool rounds	4 × web_search, 4 × fetch_url
Labeled sources	26
Tokens (all rounds)	70,782
Canary	brain_tool_calls_supported = true

What Flash called

1. web_search — Apple latest quarterly results / 10-Q
2. web_search — Q3 comparables / press release
3. web_search — apple.com newsroom, 27 June 2026
4. fetch_url — SEC EDGAR browse for CIK 0000320193, type 10-Q
5. fetch_url — filing folder 0000320193-26-000020
6. fetch_url — FY26_Q3_Consolidated_Financial_Statements.pdf (Apple Newsroom)
7. fetch_url — EDGAR index for that accession
8. web_search — “Apple Reports Third Quarter Results” 2026

What Flash answered

Q3 FY2026, quarter ended 27 June 2026, reported 31 July 2026. From Apple’s condensed consolidated statements (unaudited), millions except per share:

Metric	Q3 FY2026	Q3 FY2025
Total net sales (revenue)	109,417	94,036
Operating income	35,695	28,202
Net income	29,789	23,434
Diluted EPS	2.02	1.57

URLs it cited:

- https://www.apple.com/newsroom/pdfs/fy2026q3/FY26_Q3_Consolidated_Financial_Statements.pdf
- <https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/> (accession 0000320193-26-000020)

Control – no web

Same engine, policy: off, question “What is 2+2? Reply with the number only.” Answer: 4. No tool trace. 1.1 seconds. The plugin can stay on while a request stays offline.

5. What this means for the datacenter

today	engine :8090	→	api.deepseek.com	→	deepseek-v4-flash
later	engine :8090	→	ai-router :8080	→	same Flash, local weights

vLLM flags the Brain needs so Flash emits structured tool_calls (same JSON DeepSeek’s own API uses):

```
--tokenizer-mode deepseek_v4
--tool-call-parser deepseek_v4
--enable-auto-tool-choice
--reasoning-parser deepseek_v4
```

One cloud quirk we already handled: DeepSeek V4 thinking mode rejects a forced tool_choice (the canary and policy: required). The engine sends thinking: {type: disabled} only when the upstream host is api.deepseek.com. Local vLLM is unchanged.

Not in this run: Brave key, self-hosted SearXNG, the metal Brain on :8080. None of those block the path above.

6. How to use it

Repo: <https://github.com/phoenician-capital/local-engine-internet>. Python 3.12. You need one search key to start (SerpAPI is enough). Chat also needs DeepSeek — either the public API today, or ai-router on :8080 when the local Brain is up.

Install and start

```
git clone https://github.com/phoenician-capital/local-engine-internet.git
cd local-engine-internet
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env
# edit .env - see the table below
python -m engine
```

When it is up: toggle <http://127.0.0.1:8090/ui> · HTTP docs <http://127.0.0.1:8090/docs> · welcome JSON <http://127.0.0.1:8090/>.

What to put in .env

Variable	What you set
SERPAPI_KEY	Required to search. Enough by itself.
TAVILY_API_KEY	Optional. Longer excerpts. Joins the mix automatically.
BRAVE_API_KEY	Optional. Independent index.
UPSTREAM_LLM_BASE_URL	Chat only. Today: https://api.deepseek.com . Later: http://127.0.0.1:8080 .
UPSTREAM_API_KEY	DeepSeek API key, or ROUTER_API_KEY for ai-router. Leave empty if the Brain has no key.
UPSTREAM_MODEL	deepseek-v4-flash
PLUGIN_ENABLED	true (default). Master switch; you can also flip it at /ui.
ENGINE_API_KEY	Leave empty on your laptop. Set a secret if the port is on a network.

Do not commit .env. Never put keys in Slack or this PDF.

Search only (no model)

Use this from a pipeline, a script, or curl. The plugin must be on (default).

```
curl -s http://127.0.0.1:8090/v1/search \
-H 'Content-Type: application/json' \
-d '{"query":"Apple Inc latest 10-Q","num_results":8}'

curl -s http://127.0.0.1:8090/v1/fetch \
-H 'Content-Type: application/json' \
-d '{"url":"https://example.com"}'

curl -s http://127.0.0.1:8090/v1/research \
-H 'Content-Type: application/json' \
-d '{"query":"Apple Inc latest 10-Q","fetch_top":2}'

# Prove Mode 1:
python scripts/check_search.py --engine
```

Search returns hits, providers_used, confirmed (URLs seen by more than one index). Fetch returns page text, tier, ok. Research = search then fetch the top URLs (confirmed first).

Chat with search (the usual way)

Point any DeepSeek / vLLM client at this engine instead of api.deepseek.com. Base URL <http://127.0.0.1:8090/v1>. API key local if you left ENGINE_API_KEY empty. Model deepseek-v4-flash.

```
from openai import OpenAI
client = OpenAI(base_url="http://127.0.0.1:8090/v1", api_key="local")
r = client.chat.completions.create(
    model="deepseek-v4-flash",
```

```

    messages=[{"role": "user", "content": "What was Apple last reported quarterly revenue? Cite a live
URL."}],
)
print(r.choices[0].message.content)
print(r.model_extra.get("phoenician_tool_trace")) # or r.to_dict()

```

The snippet above uses the common Python chat client (openai package) only as an HTTP helper — the model is still Flash. DeepSeek SDK: same messages, base_url="http://127.0.0.1:8090" (no /v1). Cursor / Continue: custom host http://127.0.0.1:8090/v1, model deepseek-v4-flash.

Optional extra on the JSON body (the engine strips it before Flash sees it):

```

{
  "phoenician_web": {
    "policy": "auto"
  }
}

```

policy	When to use it
auto (default)	Flash decides. "What is 2+2?" stays offline. A dated company question searches.
required	Research desks. Must complete a successful search, or HTTP 424.
off	This request only – no tools. Needed for stream: true.

If the reply JSON includes phoenician_tool_trace, Flash searched or fetched. phoenician_sources lists the URLs. Treat a tool result that starts with ERROR: as a hard failure — do not treat it as a fact.

Turn the internet off for everyone at /ui or POST /v1/plugin {"enabled": false}. Chat then becomes a plain pass-through to Flash. One request can turn it off with header X-Phoenician-Plugin: off.

Smoke test and the \$4 prompt

```

python scripts/smoke.py

curl -s http://127.0.0.1:8090/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{
  "model": "deepseek-v4-flash",
  "messages": [{"role": "user", "content": "Apple Inc (AAPL) most recently reported quarterly revenue,
operating income, and diluted EPS. Prefer the official 10-Q or earnings release on SEC EDGAR or
investor.apple.com. Return the filing URL."}],
  "phoenician_web": {"policy": "required"}
}'

```

Smoke needs the engine and Flash (API or local Brain). It asks a dated Apple-revenue question with policy: required, then "2+2" with policy: off. Pass = a real URL in the sources, and no tools on the math question. In the curl reply, look for phoenician_tool_trace and phoenician_sources. Full file map of the repo: Local-LLM-Internet-Engine-Method.pdf. Neither PDF contains API keys.